

9.a. Multi – level Linked List
–Print List(Main List and Child)
–Time Complexity – Analysis – Part 1

Program:

```
void printList(Node *node, int level = 0)
{
    while (node != nullptr)
    {
        for (int i = 0; i < level; i++)
            cout << "    ";
        cout << "|-- " << node->data << endl;

        if (node->child != nullptr)
        {
            printList(node->child, level + 1);
        }
        node = node->next;
    }
}

...

case 9:
    cout << "Current List Structure: " << endl;
    printList(head);
    break;
```

Time Complexity Analysis

$$T(\text{node}, l) = O(l) + T(\text{node} \rightarrow \text{child}, l + 1) + T(\text{node} \rightarrow \text{next}, l)$$

Where:

- *$O(l)$ represents the inner for loop, which prints exactly l spaces.*
- *$T(\text{node} \rightarrow \text{child}, l + 1)$ is the recursive call for the child level.*
- *$T(\text{node} \rightarrow \text{next}, l)$ is the continuation of the while loop for the next node at the current level.*

A more practical way to look at the general equation is to consider the structure as a whole. Assuming the data structure has no cycles, this algorithm visits every single node exactly once.

Let N be the total number of nodes in the entire multi – level data structure, and let d_i be the depth (level) of the i – th node. The total time T is the sum of the work done at each individual node:

$$T(N) = \sum_{i=1}^N (c_1 \times d_i + c_2)$$

What the variables mean:

- *$T(N)$: The total time (work) the program takes for N nodes.*
- *$\sum_{i=1}^N$: This is a summation symbol. It means "add up the work done at every single node, starting from node 1 up to node N ."*
- *i : The specific node we are currently looking at (1st node, 2nd node, etc.).*
- *d_i : The depth (or level) of that specific i – th node.*
- *c_1 : The constant time it takes to print a single space in your for loop. So, $c_1 \cdot d_i$ is the total time spent in the for loop for that node.*
- *c_2 : The constant time for everything else at that node*

(checking `node != nullptr`, printing the actual data, making the recursive call).

Edge Case vs. Best Case

When computer scientists talk about the Best, Worst, and Average cases, they are generally analyzing how the algorithm behaves as the input size (N) grows arbitrarily large ($N > 0$). They are answering the question: "Given a specific dataset of size N , what structural arrangement of that data results in the least or most work?"

- *The Trivial Edge Case ($N = 0$): If the input size is zero (a `nullptr`), the algorithm takes $O(1)$ time. This is an edge case.*

This is often called:

- *Empty – case*
- *Null – case*
- *Trivial case*
- *Edge case*

Some authors also informally call it the absolute best case, because no input data exists and the algorithm performs the minimum possible work.

- *The Asymptotic Best Case ($N > 0$): If we have N nodes, what is the best possible way those N nodes could be arranged? The best possible arrangement for this specific algorithm is a completely flat list with no child pointers. It still has to visit all N nodes once, resulting in an $O(N)$ time complexity.*

When Best/Worst/Average Case discussing Complexity,

we normally assume:

$$N > 0$$

and ask: "Among all inputs of size N , which one requires the least work!"

Final Big – O Time Complexity

Because the time spent at each node depends entirely on its level, the final

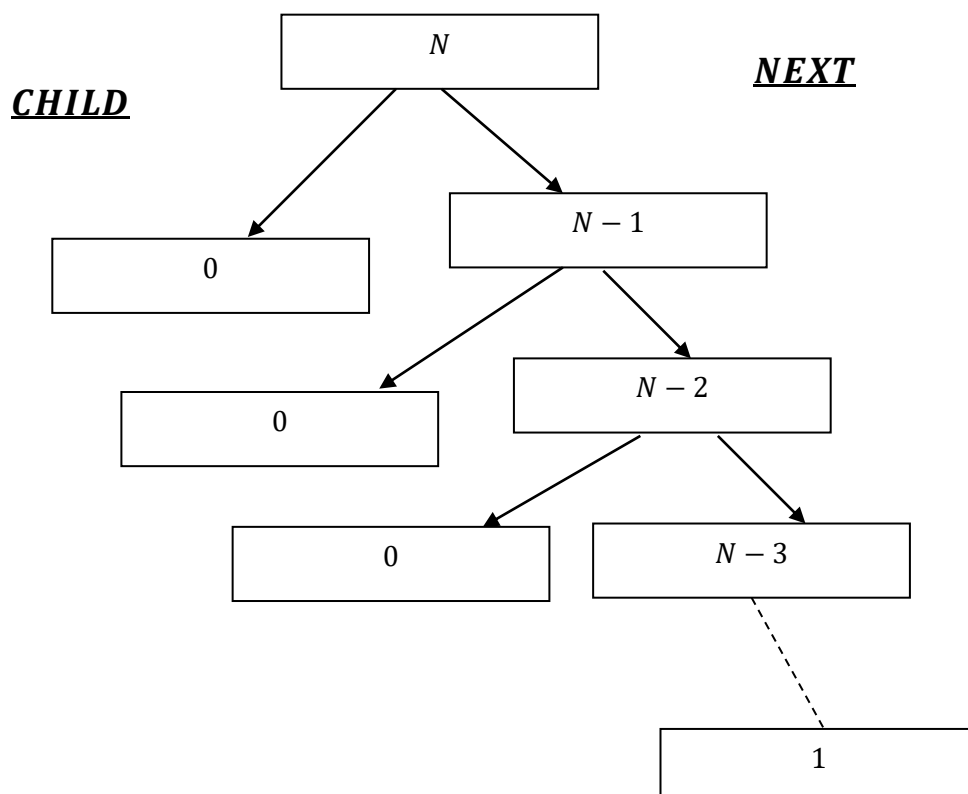
Big – O complexity varies based on the shape of the data structure.

- ***General Upper Bound: If N is the total number of nodes and L is the maximum depth (the highest value of level), the general time complexity is:***

$$O(N \cdot L)$$

- ***Best Case (Flat List):***

If there are no child pointers at all, level is always 0. The for loop never executes, and the equation simplifies to visiting N nodes in constant time. That is:

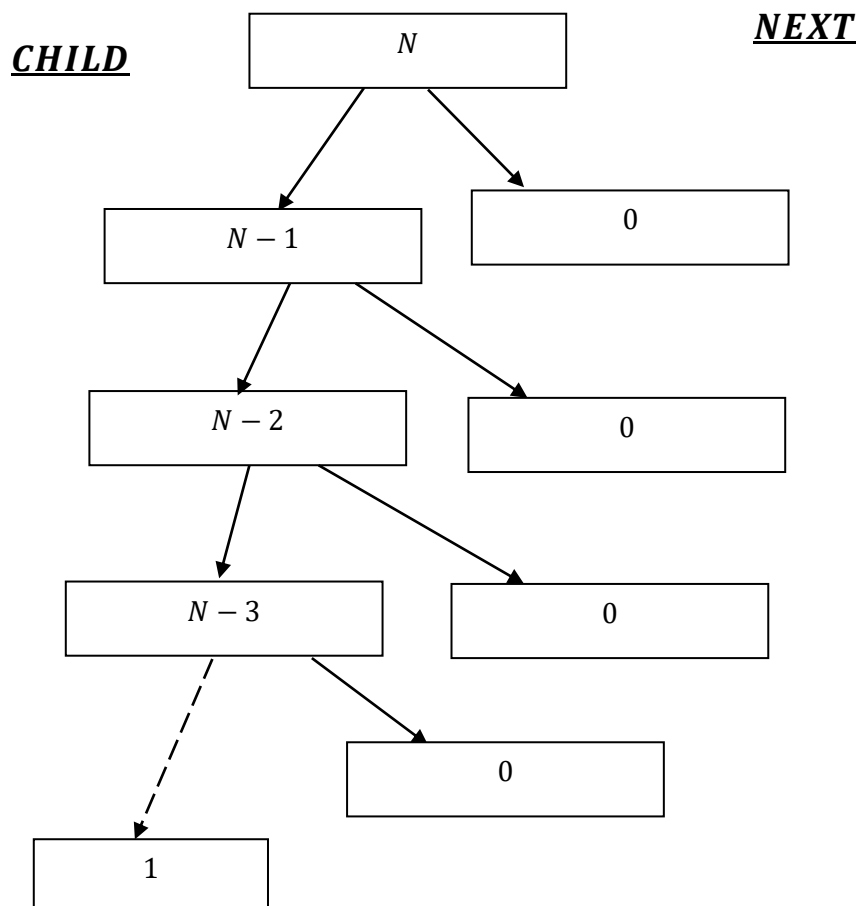


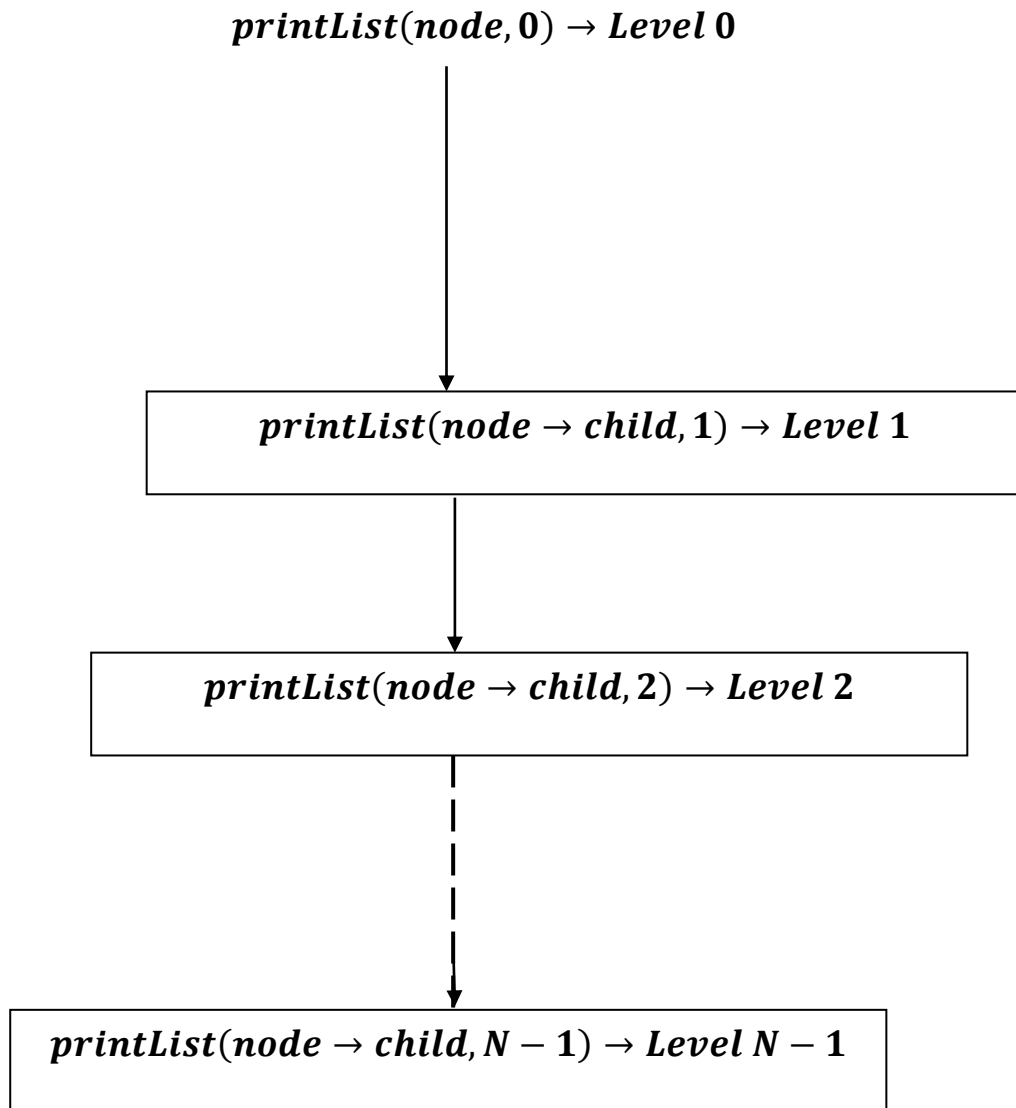
If every single node is a next of the previous node ,Level will always be 0 ,while loop will get executed,same stack frame used ,child always nullptr.for loop never gets executed.

Time Complexity: $O(N)$

- **Worst Case (Deeply Nested List):**

If every single node is a child of the previous node (a straight vertical line where next is always nullptr), the levels will be 0,1,2, ..., $N - 1$. The total work becomes the sum of the first N integers:





$$T(N) = \sum_{i=0}^{N-1} i = \frac{N(N-1)}{2}$$

Hence, Worst Case Time Complexity = $O(N^2)$

The Worst Case Scenario (Table)

In the worst case, you have a completely vertical, deeply nested list.

- ***The 1st node is at level = 0.***
- ***The 2nd node is a child of the 1st, so it is at level = 1.***
- ***The 3rd node is a child of the 2nd, so it is at level = 2.***
- ***This pattern continues until the $N - \text{th}$ node, which will be at level = $N - 1$.***

Let's look at a table showing the exact amount of work done at each node:

<i>Node Number</i> <i>(i)</i>	<i>Node Level</i> <i>(d_i)</i>	<i>Time in for loop</i> <i>$c_1 \times d_i$</i>	<i>Constant</i> <i>Work</i> <i>(c_2)</i>	<i>Total</i> <i>Work</i> <i>($c_1 \times d_i + c_2$)</i>
1	0	$c_1 \times 0 = 0$	c_2	$0 + c_2$
2	1	$c_1 \times 1 = c_1$	c_2	$c_1 + c_2$
3	2	$c_1 \times 2 = 2c_1$	c_2	$2c_1 + c_2$
4	3	$c_1 \times 3 = 3c_1$	c_2	$3c_1 + c_2$
....				
N	$N - 1$	$c_1 \times (N - 1)$	c_2	$c_1 \times (N - 1)$ $+c_2$

Total Work:

$$= (0 + c_2) + (c_1 + c_2) + (2c_1 + c_2) + (3c_1 + c_2) + \dots + (c_1 \times (N - 1) + c_2)$$

$$= [0 + c_1 + 2c_1 + 3c_1 + \dots + c_1 \times (N - 1)] + (c_2 + c_2 + \dots N \text{ times})$$

$$= c_1[0 + 1 + 2 + 3 + \dots + N - 1] + (c_2 \times N)$$

$$= c_1 \left[\sum_{i=0}^{N-1} i \right] + (c_2 \times N)$$

$$= c_1 \left[a_0 + \sum_{i=1}^{N-1} i \right] + (c_2 \times N)$$

$$= c_1 \left[0 + \frac{N(N-1)}{2} \right] + (c_2 \times N)$$

$$= c_1 \left[\frac{N^2 - N}{2} \right] + (c_2 \times N)$$

$$= c_1 \left[\frac{N^2}{2} - \frac{N}{2} \right] + (c_2 \times N)$$

$$= c_1 \times \frac{N^2}{2} - c_1 \times \frac{N}{2} + (c_2 \times N)$$

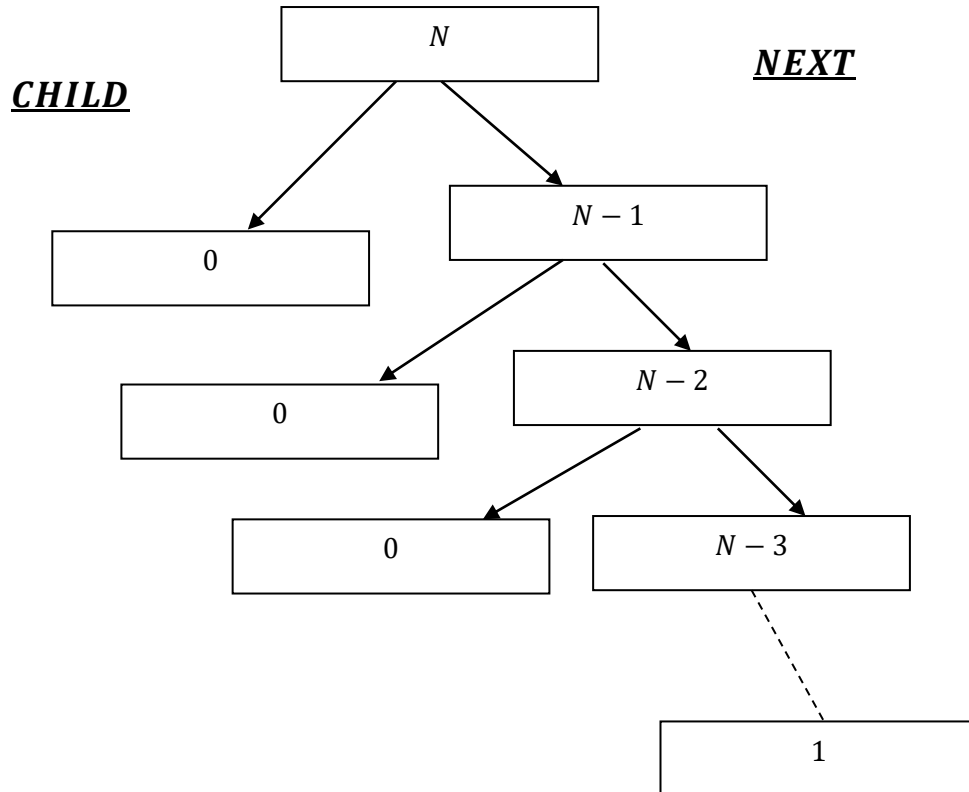
Applying Big – O, we get:

$$= O \left(c_1 \times \frac{N^2}{2} - c_1 \times \frac{N}{2} + (c_2 \times N) \right)$$

$$= O(N^2)$$

Note, Recursion Works on , the hidden tree structure:

Imbalanced (Flat)



Here ,while loop runs but for loop never runs.hence,

Recurrence Relation:

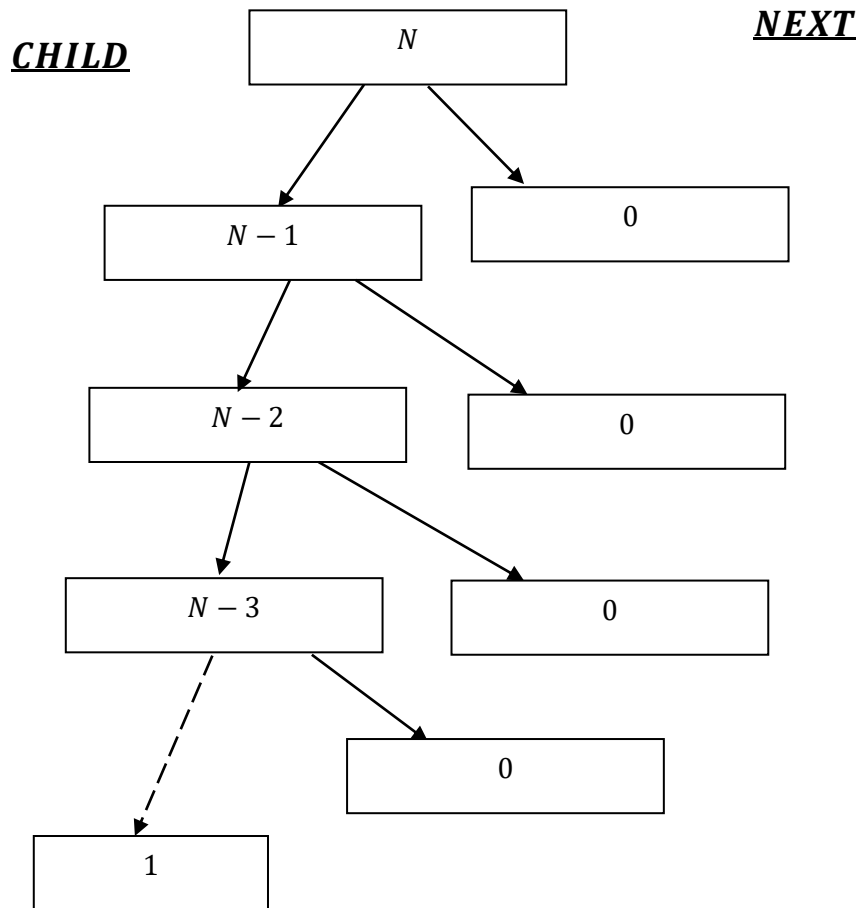
$$T(n) = \begin{cases} 1 & \text{for } n = 0 \\ T(N - 1) + c & \text{for } n > 0 \end{cases}$$

Here Work done at Node is `c` i.e. $O(1)$.

Recurrence Table

<i>Step/ Depth(k)</i>	<i>Nodes Remaining (i)</i>	<i>Node's Level (d_i)</i>	<i>Time in for loop $c_1 \times d_i$</i>	<i>Constant Work (c₂)</i>	<i>Total Work ($c_1 \times d_i + c_2$)</i>	<i>Current Equation</i>
1	<i>N</i>	0	$c_1 \times 0 = 0$	c_2	$0 + c_2 = c_2$	$T(N) = T(N-1) + c$
2	$N - 1$	0	$c_1 \times 0 = 0$	c_2	$0 + c_2 = c_2$	$T(N-1) = T(N-2) + c$
3	$N - 2$	0	$c_1 \times 0 = 0$	c_2	$0 + c_2 = c_2$	$T(N-2) = T(N-3) + c$
....						
<i>N</i>	1	0	$c_1 \times 0 = 0$	c_2	$0 + c_2 = c_2$	$T(1) = T(0) + c$
<i>Base Case</i>	0	<i>N/A</i>	<i>N/A</i>	c_2	$0 + c_2 = c_2$	$T(0) = c$

Imbalanced (Deep)



Here ,while loop runs ,for loop runs.hence,

Recurrence Relation:

$$T(n) = \begin{cases} 1 & \text{for } n = 0 \\ T(N-1) + c \times N & \text{for } n > 0 \end{cases}$$

Recurrence Table

Step/ Depth(k)	Nodes Remaining (i)	Node's Level (d_i)	Time in for loop $c_1 \times d_i$	Constant Work (c₂)	Total Work ($c_1 \times d_i + c_2$)	Current Equation
1	N	0	$c_1 \times 0 = 0$	c₂	0 + c₂	$T(N) = T(N - 1) + c_2$
2	N - 1	1	$c_1 \times 1 = c_1$	c₂	c₁ + c₂	$T(N - 1) = T(N - 2) + (c_1 + c_2)$
3	N - 2	2	$c_1 \times 2 = 2c_1$	c₂	2c₁ + c₂	$T(N - 2) = T(N - 3) + (2c_1 + c_2)$
....						
N	1	N - 1	$c_1 \times (N - 1)$	c₂	$c_1 \times (N - 1) + c_2$	$T(1) = T(0) + (c_1 \times (N - 1) + c_2)$
Base Case	0	N/A	N/A	c₂	0 + c₂ = c₂	$T(0) = c$

Perfect Balanced

$\frac{T(N-1)}{2}$ *for Next Nodes.*

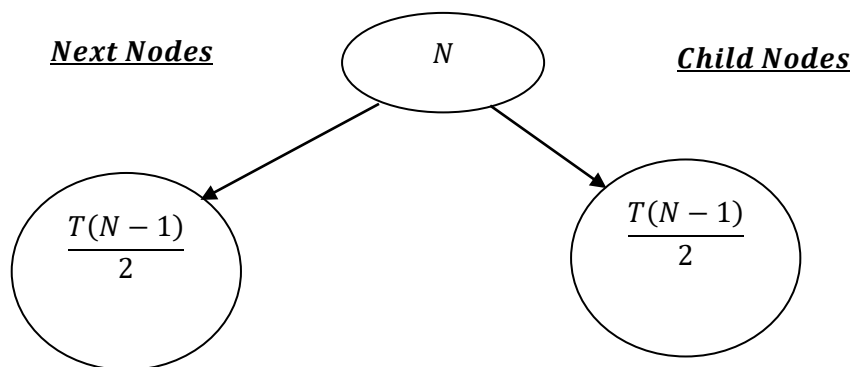
$\frac{T(N-1)}{2}$ *for child Nodes*

`c` → constant or $O(1)$.

While equations perfectly describe how the algorithm traverses the nodes, there is one crucial detail specific to your printList program: the "c" is not always a constant.

In standard recurrence relations, c represents $O(1)$ constant time work.

In the code, the work at current node includes a for loop that prints spaces based on the level.



Work Done by CPU per Node or per Problem Size

Printing:

```
cout << "    ";  
cout << "|-- " << node->data << endl;
```

And Condition checking :

```
while (node != nullptr)  
    for (int i = 0; i < level; i++)  
        if (node->child != nullptr)
```

Condition Checking

And Initialization :

```
node = node->next;  
for (int i = 0; i < level; i++)
```

Initialization

```
printList(node->child, level + 1);
```

Pushing the stack.

This all take constant time `c` represented as $O(1)$.

$$T(n) = \frac{T(N-1)}{2} + \frac{T(N-1)}{2} + O(1) \text{ or } c$$

$$T(n) = T\left(\frac{N-1}{2}\right) + T\left(\frac{N-1}{2}\right) + c$$

$$= 2T\left(\frac{N-1}{2}\right) + c$$

$$T(n) = \begin{cases} 1 & \text{for } n = 0 \\ 2T\left(\frac{N-1}{2}\right) + c & \text{for } n > 0 \end{cases}$$
